

Research Notes on Sparse attention transformer

Sparse attention transformer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

>> Section 1

Multimodality Transformers can also be used/adapted for modalities (input or output) beyond just text, usually by finding a way to "tokenize" the modality. Multimodal models can either be trained from scratch, or by finetuning. A 2022 study found that transformers pretrained only on natural language can be finetuned on only 0.03% of parameters and become competitive with LSTMs on a variety of logical and visual tasks, demonstrating transfer learning. The LLaVA was a vision-language model composed of a language model (Vicuna-13B) and a vision model (ViT-L/14), connected by a linear layer. Only the linear layer is finetuned. Vision transformers adapt the transformer to computer vision by breaking down input images as a series of patches, turning them into vectors, and treating them like embedding vector of tokens in a standard transformer. Conformer and later Whisper follow the same pattern for speech recognition, first turning the speech signal into a spectrogram, which is then treated like an image, i.e. broken down into a series of patches, turned into vectors and treated like embedding vector of tokens in a standard transformer. Perceivers are a variant of transformers designed for multimodality. For image generation, notable architectures are DALL-E 1 (2021), Parti (2022), Phenaki (2023), and Muse (2023). Unlike later models, DALL-E is not a diffusion model. Instead, it uses a decoder-only transformer that autoregressively generates a text, followed by the token representation of an image, which is then converted by a variational autoencoder to an image. Parti is an encoder-decoder transformer, where the encoder processes a text prompt, and the decoder generates a token representation of an image. Muse is an encoder-only transformer that is trained to predict masked image tokens from unmasked image tokens. During generation, all input tokens are masked, and the highest-confidence predictions are included for the next iteration, until all tokens are predicted. Phenaki is a text-to-video model. It is a bidirectional masked transformer conditioned on pre-computed text tokens. The generated tokens are then decoded to a video.

>> Section 2

Alternative normalizations The normalization used in the transformer can be different from LayerNorm. One example is RMSNorm which is used in the Llama series. Other examples include ScaleNorm and FixNorm.

>> Section 3

```
/* second sublayer */ z_e_copy ← copy(z_e) for each t in
1:length(z_e) do z_e[t] ← layer.layer_norm(z_e[t]) z_e ←
layer.feedforward(z_e) for each t in 1:length(z_e) do z_e[t]
← z_e[t] + z_e_copy[t]
```

>> Section 4

```
/* third sublayer */ z_d_copy ← copy(z_d) for each t in
1:length(z_d) do z_d[t] ← layer.layer_norm(z_d[t]) z_d ←
layer.feedforward(z_d) for each t in 1:length(z_d) do z_d[t]
← z_d[t] + z_d_copy[t]
```

>> Section 5

Alternative activation functions The original transformer uses ReLU activation function. Other activation functions were developed. The Llama series and PaLM used SwiGLU; both GPT-1 and BERT used GELU. Alternative activation functions are often used in combination with Gated Linear Units in the feedforward module.

>> Section 6

FlashAttention FlashAttention is an algorithm that implements the transformer attention mechanism efficiently on a GPU. It is a communication-avoiding algorithm that performs matrix multiplications in blocks, such that each block fits within the cache of a GPU, and by careful management of the blocks it minimizes data copying between GPU caches (as data movement is slow). See the page on softmax for details. An improved version, FlashAttention-2, was developed to cater to the rising demand for language models capable of handling longer context lengths. It offers enhancements in work partitioning and parallelism, enabling it to achieve up to 230 TFLOPs/s on A100 GPUs (FP16/BF16), a 2x speed increase over the original FlashAttention. Key advancements in FlashAttention-2 include the reduction of non-matmul FLOPs, improved parallelism over the sequence length dimension, better work partitioning between GPU warps, and added support for head dimensions up to 256 and multi-query attention (MQA) and grouped-query attention (GQA). Benchmarks revealed FlashAttention-2 to be up to 2x faster than FlashAttention and up to 9x faster than a standard attention implementation in PyTorch. Future developments include optimization for new hardware like H100 GPUs and new data types like FP8. FlashAttention-4 focuses on pipelining to increase instruction throughput, and was developed to perform particularly well on Blackwell GPUs.

>> Section 7

Speculative decoding is a method to accelerate token decoding. Similarly to speculative execution in CPUs, future tokens are computed quickly, then verified. If the quickly computed tokens are incorrect, they are discarded

Research Notes on Sparse attention transformer

Sparse attention transformer

and computed slowly. The key factor in speculative decoding is that a transformer decoder can verify faster than it can decode, in the following sense. Suppose we have two transformer models like GPT-3 and GPT-3-small, both with a context window size of 512. To generate an entire context window autoregressively with greedy decoding with GPT-3, it must be run for 512 times, each time generating a token $x_1, x_2, \dots, x_{512}$, taking time $512 T_{\text{GPT-3}}$. However, if we had some educated guess for the values of these tokens, we could verify all of them in parallel, in one run of the model, by checking that each x_t is indeed the token with the largest log-likelihood in the t -th output. In speculative decoding, a smaller model or some other simple heuristic is used to generate a few speculative tokens that are subsequently verified by the larger model. For example, suppose we use GPT-3-small to generate four speculative tokens: $\tilde{x}_1, \tilde{x}_2, \tilde{x}_3, \tilde{x}_4$. This only takes $4 T_{\text{GPT-3-small}}$. These tokens are then run through the larger GPT-3 in one go. Suppose that $\tilde{x}_1$ and $\tilde{x}_2$ are verified by GPT-3 as what it would have picked, then those are kept, but $\tilde{x}_3$ is not, so $\tilde{x}_3, \tilde{x}_4$ are discarded, and GPT-3 is run on those. This would take $4 T_{\text{GPT-3-small}} + 3 T_{\text{GPT-3}}$, which might be shorter than $4 T_{\text{GPT-3}}$. For non-greedy decoding, similar ideas apply, except the speculative tokens are accepted or rejected stochastically, in a way that guarantees the final output distribution is the same as if speculative decoding was not used.